

প্রকল্পের সারাংশ (Project Overview)

এই অ্যাপ্লিকেশনটি একটি ফিটনেস ট্র্যাকিং অ্যাপ যেখানে ব্যবহারকারীরা:

- workouts লগ করতে পারে
- steps ট্র্যাক করতে পারে
- খাবারের ক্যালোরি ট্র্যাক করতে পারে
- AI চ্যাটবটের সাথে কথা বলতে পারে
- অ্যাডমিন ড্যাশবোর্ড দেখতে পারে

ব্যবহৃত টেকনোলজিস:

- Flutter (UI)
- Firebase (Authentication + Firestore)
- Google Gemini AI (চ্যাটবট)
- Provider (State Management)
- Shared Preferences (Local Storage)

File-by-File ব্যাখ্যা (Bangla)

1. admin_dashboard_screen.dart

উদ্দেশ্য: অ্যাডমিন প্যানেল যেখানে সিস্টেমের পরিসংখ্যান দেখানো হয়।

লাইন বাই লাইন ব্যাখ্যা:

```
dart
class AdminDashboardScreen extends StatelessWidget {
```

- **StatelessWidget** ব্যবহার করা হয়েছে কারণ এটি স্টেট পরিবর্তন করে না, শুধু ডেটা দেখায়।

```
dart
final firestoreService = Provider.of<FirestoreService>(context, listen: false);
```

- **Provider** থেকে FirestoreService নেওয়া হয়েছে। `listen: false` দিয়ে বলা হয়েছে যে এই widget টি পরিবর্তন শুনবে না, শুধু একবার ডেটা নেবে।

```
dart
StreamBuilder<List<UserModel>>(  
  stream: firestoreService.getAllUsers(),  
  builder: (context, snapshot) {  
    final userCount = snapshot.data?.length ?? 0;  
    return _buildStatCard(...);  
  },  
)
```

- **StreamBuilder** ব্যবহার করা হয়েছে কারণ Firestore থেকে ডেটা আসতে সময় লাগে। এটি রিয়েল-টাইম আপডেট দেয়।
- `getAllUsers()` একটি stream রিটার্ন করে, যাতে নতুন ইউজার যোগ হলে অটোমেটিক UI আপডেট হয়।

```
dart
FutureBuilder<int>(  
  future: firestoreService.getTotalWorkouts(),  
  builder: (context, snapshot) {  
    final workoutCount = snapshot.data ?? 0;  
    return _buildStatCard(...);  
  },  
)
```

- **FutureBuilder** ব্যবহার করা হয়েছে কারণ `getTotalWorkouts()` একটি Future রিটার্ন করে (একবার ডেটা নিয়ে আসে)।

কুই: কেন StreamBuilder আর FutureBuilder আলাদা?

- **StreamBuilder:** রিয়েল-টাইম আপডেটের জন্য (যেমন ইউজার লিস্ট)
- **FutureBuilder:** একবার ডেটা আনার জন্য (যেমন টোটাল ওয়ার্কআউট কাউন্ট)

প্রশ্ন আসতে পারে:

- "কেন অ্যাডমিন ড্যাশবোর্ডে FutureBuilder আর StreamBuilder দুইটাই ব্যবহার করেছো?"
- "getAllUsers() stream কেন? এতে কি performance issue হয়?"

2. chat_history_screen.dart

উদ্দেশ্য: ইউজারের চ্যাট হিস্ট্রি দেখানো এবং ডিলিট করার অপশন দেওয়া।

```
dart
return Dismissible(
  key: Key(log.id),
  direction: DismissDirection.endToStart,
  onDismissed: (_) {
    firestoreService.deleteChatLog(user.uid, log.id);
  },
  child: ...
)
```

- **Dismissible** ব্যবহার করা হয়েছে যাতে ইউজার swipe করে চ্যাট ডিলিট করতে পারে।
- `key: Key(log.id)` দিয়ে প্রতিটি আইটেমকে ইউনিক আইডি দেওয়া হয়েছে।

```
dart
void _showClearHistoryDialog(BuildContext context, String uid, FirestoreService service) {
  showDialog(...)
}
```

- কনফার্মেশন ডায়ালগ দেখানো হয়েছে যাতে ইউজার ভুলবশত সব চ্যাট ডিলিট না করে।

কুই: Dismissible এর key কেন লাগে?

- Flutter কে বুঝতে হয় কোন আইটেম ডিলিট হয়েছে, তাই unique key প্রয়োজন।

প্রশ্ন আসতে পারে:

- "ব্যাচ ডিলিট করলে performance কীভাবে improve করবে?"

- "Firestore ব্যচ অপারেশন কী?"

3. chatbot_screen.dart

উদ্দেশ্য: AI চ্যাটবট যেখানে ইউজার প্রশ্ন করতে পারে।

```
dart
final GeminiService _geminiService = GeminiService();
```

- **GeminiService** ইনস্ট্যান্স তৈরি করা হয়েছে AI এর সাথে কথা বলার জন্য।

```
dart
void _sendMessage() async {
  final response = await _geminiService.sendMessage(userMessage.text);
  // Save to Firestore
  final chatLog = ChatLogModel(...);
  firestore.saveChatLog(chatLog);
}
```

- ইউজারের মেসেজ AI তে পাঠানো হয়, রেসপন্স আসলে তা UI তে দেখানো হয় এবং Firestore এ সেভ করা হয়।

```
dart
GestureDetector(
  onTap: () {
    _geminiService.resetChat();
    setState(() {
      _messages.clear();
      _messages.add(welcomeMessage);
    });
  },
)
```

- রিসেট বাটনে পুরোনো চ্যাট ক্লিয়ার করে নতুন সেশন শুরু হয়।

কুই: GeminiService কেন আলাদা class এ রাখা হয়েছে?

- **Separation of Concern:** UI থেকে AI logic আলাদা রাখা ভালো প্র্যাকটিস।

প্রশ্ন আসতে পারে:

- "Gemini API key কোথায় রাখা উচিত?"
- "API call ব্যর্থ হলে কী হয়?"
- "চ্যাট হিস্ট্রি কেন Firestore এ সেভ করছো? লোকাল স্টোরেজ না?"

4. add_step_form.dart

উদ্দেশ্য: স্টেপ লগ করার ফর্ম।

```
dart
int get _steps => int.tryParse(controller.text) ?? 0;
double get _estimatedCalories => _steps * 0.04;
```

- **Getter method** ব্যবহার করে ক্যালোরি ক্যালকুলেট করা হচ্ছে।

```
dart
ElevatedButton(
  onPressed: _steps > 0 ? () async {
    final log = StepLogModel(...);
    await widget.service.logSteps(log);
    Navigator.pop(context);
  } : null,
)
```

- স্টেপ > 0 হলে বাটন সক্রিয় হয়।

কুই: কেন `int.tryParse()` ব্যবহার করেছো?

- কারণ ইউজার ভ্যালিড ইনপুট দিতে পারে, `tryParse` null return করে তাই safe.

প্রশ্ন আসতে পারে:

- "ইউজার যদি ০ বা নেগেটিভ স্টেপ দেয়?"
- "ক্যালোরি ক্যালকুলেশন কীভাবে কাজ করে?"

5. steps_screen.dart

উদ্দেশ্য: স্টেপ ট্র্যাকারের মূল UI।

```
dart
class _CircularProgressPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    canvas.drawArc(...);
  }
}
```

- কাস্টম **CustomPainter** ব্যবহার করে সার্কুলার প্রোগ্রেস বার তৈরি করা হয়েছে।

```
dart
StreamBuilder<int>(
  stream: firestoreService.getTodaySteps(user.uid),
  builder: (context, snapshot) {
    final todaySteps = snapshot.data ?? 0;
    final progress = (todaySteps / _stepGoal).clamp(0.0, 1.0);
    // Animated progress
  },
)
```

- Today's steps রিয়েল-টাইম আপডেট পেতে StreamBuilder ব্যবহার করা হয়েছে।

```
dart
Dismissible(
  onDismissed: (direction) {
    firestoreService.deleteStepLog(user.uid, log.id);
  },
)
```

- স্টেপ লগও swipe করে ডিলিট করা যায়।

কুই: CustomPainter কেন ব্যবহার করেছো?

- Flutter এর built-in CircularProgressIndicator ছিল না কাস্টম ডিজাইনের জন্য, তাই নিজে ঐঁকেছি।

প্রশ্ন আসতে পারে:

- "AnimationController কেন ব্যবহার করেছো?"
- "কেন StreamBuilder এ initialData: 0 দিয়েছো?"

6. chat_log_model.dart

উদ্দেশ্য: চ্যাট লগের ডেটা মডেল।

```
dart
Map<String, dynamic> toMap() {
  return {
    'timestamp': Timestamp.fromDate(timestamp),
  };
}
```

- DateTime কে Firestore Timestamp এ কনভার্ট করা হয়েছে।

কুই: কেন Timestamp ব্যবহার করেছো?

- Firestore DateTime সাপোর্ট করে না, তাই Timestamp ব্যবহার করতে হয়।

7. chat_message_model.dart

উদ্দেশ্য: চ্যাট মেসেজের UI মডেল।

```
dart
String get formattedTime => DateFormat('HH:mm').format(timestamp);
```

- Getter ব্যবহার করে সময় ফরম্যাট করা হয়েছে।

8. step_log_model.dart

উদ্দেশ্য: স্টেপ লগের ডেটা মডেল।

```
dart
double get caloriesBurned => steps * 0.04;
```

- প্রতি স্টেপে 0.04 ক্যালোরি বার্ন হয় (একটি সাধারণ অনুমান)।

9. firestore_service.dart

উদ্দেশ্য: Firestore এর সাথে সব CRUD অপারেশন।

```
dart
Stream<List<UserModel>> getAllUsers() {
  return _db.collection('users')
    .snapshots()
    .map((snapshot) => snapshot.docs.map((doc) => UserModel.fromMap(doc.data())).toList());
}
```

- .snapshots() রিয়েল-টাইম আপডেট দেয়।

```
dart
Future<int> getTotalWorkouts() async {
  int total = 0;
  final users = await _db.collection('users').get();
  for (var user in users.docs) {
```

```

    final workouts = await _db.collection('users').doc(user.id).collection('workouts')
    ).get();
    total += workouts.docs.length;
  }
  return total;
}

```

- সব ইউজারের ওয়ার্কআউট কাউন্ট করে (N+1 Query Problem)।

কুই: getTotalWorkouts() এ কী সমস্যা?

- N+1 Query Problem: প্রথমে সব ইউজার আনে, তারপর প্রতিটি ইউজারের জন্য আলাদা কুয়েরি করে।
- **Solution:** Firebase এর aggregation queries বা sub-collections এ counter রাখা।

প্রশ্ন আসতে পারে:

- "কেন .snapshots() ব্যবহার করেছো?"
- "Batch write কী?"
- "Security Rules কিভাবে সেট করবে?"

10. gemini_service.dart

উদ্দেশ্য: Gemini AI এর সাথে যোগাযোগ।

```

dart
GeminiService() {
  _model = GenerativeModel(
    model: 'gemini-2.5-flash',
    apiKey: _apiKey,
    generationConfig: GenerationConfig(
      temperature: 0.7,
      topK: 40,
      topP: 0.95,

```

```

        maxOutputTokens: 1024,
      ),
    );
  }

```

- **temperature:** 0.7 (কিছুটা ক্রিয়েটিভ কিন্তু কনসিস্টেন্ট)
- **maxOutputTokens:** 1024 (রেসপন্সের দৈর্ঘ্য সীমিত)

```

dart
Future<String> sendMessage(String message) async {
  final response = await _chat.sendMessage(content);
  return response.text ?? 'Error';
}

```

কুই: API Key কীভাবে সিকিউর করবে?

- **উত্তর:** Firebase Functions বা backend এ রাখা উচিত, client এ না।

প্রশ্ন আসতে পারে:

- "Gemini API এর rate limit কীভাবে হ্যান্ডেল করবে?"
- "Error handling কীভাবে করেছে?"

11. notification_service.dart

উদ্দেশ্য: নোটিফিকেশন দেখানো।

```

dart
tz.TZDateTime _nextInstanceOf8AM() {
  final now = tz.TZDateTime.now(tz.local);
  return tz.TZDateTime(tz.local, now.year, now.month, now.day, 8);
}

```

- সময় অঞ্চল (timezone) ব্যবহার করে সকাল ৮টার সময় বের করা হয়েছে।

কুই: কেন timezone package ব্যবহার করেছে?

- সঠিক লোকাল টাইমে নোটিফিকেশন শিডিউল করতে।

12. step_service.dart

উদ্দেশ্য: স্টেপ ম্যানেজ করার সার্ভিস।

```
dart
Future<void> updateStepsManually(int steps) async {
  await _firestoreService.updateTodayStepLog(_userId, _todaySteps);
  final prefs = await SharedPreferences.getInstance();
  await prefs.setInt('todaySteps', steps);
}
```

- Firestore + Local Storage (SharedPreferences) উভয় জায়গায় স্টেপ সেভ করে।

কুই: কেন SharedPreferences ব্যবহার করেছো?

- অফলাইন সাপোর্ট এবং দ্রুত ডেটা অ্যাক্সেসের জন্য।

13. firebase_options.dart

উদ্দেশ্য: Firebase কনফিগারেশন।

```
dart
static const FirebaseOptions web = FirebaseOptions(
  apiKey: 'AIzaSyCvnfSDAwZKqjBSvaqdNN0j4u4DBNLag80',
  appId: '1:328881203269:web:5a096f9e8a76377554db2c',
  projectId: 'fit-axis-web',
);
```

- Web এবং Android এর জন্য আলাদা config.

কুই: Web আর Android এর config আলাদা কেন?

- Firebase এ different app entries থাকে, তাই আলাদা credentials.

প্রকল্পের Flow (কীভাবে কাজ করে)

1. **ইউজার লগইন করে** → Firebase Authentication
2. **হোম স্ক্রিনে** → workouts, steps, food, water দেখায়
3. **Workout যোগ করে** → Firestore এ সেভ হয় → UI আপডেট হয় (Stream)
4. **Steps ট্র্যাক করে** → Add Step Form → Firestore + SharedPreferences
5. **Chatbot এ কথা বলে** → Gemini API → রেসপন্স দেখায় → Firestore এ সেভ হয়
6. **অ্যাডমিন ড্যাশবোর্ডে** → সব ইউজারের ডেটা দেখায়

ভাইভাতে সম্ভাব্য প্রশ্ন ও উত্তর

1. Architecture Related

Q: কেন Provider ব্যবহার করেছো?

- **A:** হালকা ওজনের state management, dependency injection সহজ, এবং অ্যাপ ছোট তাই Provider যথেষ্ট।

Q: Bloc বা Riverpod ব্যবহার করতে পারতে?

- **A:** পারতাম, কিন্তু Provider সহজ এবং আমাদের প্রয়োজন মেটায়।

2. Firebase Related

Q: Firestore এ কীভাবে ডেটা মডেল করেছো?

- **A:** Users → sub-collections (workouts, steps, food, water, chats)
- প্রতিটি ইউজারের ডকুমেন্টের下有 collection.

Q: Security Rules কীভাবে সেট করবে?

- **A:** শুধু authenticated ইউজার তার নিজের ডেটা read/write করতে পারবে। অ্যাডমিন সব ডেটা দেখতে পারবে।

```
javascript
match /users/{userId}/{document=**} {
  allow read, write: if request.auth.uid == userId;
}
```

3. Performance Related

Q: StreamBuilder বারবার ডেটা আনে, performance issue হবে না?

- **A:** Firestore smartly sync করে, শুধু পরিবর্তিত ডেটা আনে। তবে অনেক ডেটা হলে pagination ব্যবহার করতে হবে।

Q: কীভাবে optimize করবে?

- **A:**
 - `.limit()` ব্যবহার করে ডেটা সীমিত করা
 - Indexing
 - Aggregation queries

4. API & AI Related

Q: Gemini API এর key client side এ রাখা কি safe?

- **A:** না, এটি unsafe। Production এ backend (Firebase Functions) ব্যবহার করা উচিত।

Q: API fail হলে কী হয়?

- **A:** try-catch করে error message দেখাই এবং retry mechanism যোগ করা যেতে পারে।

5. State Management

Q: setState() ব্যবহার করেছো, এতে কী সমস্যা?

- **A:** ছোট widget এর জন্য ঠিক আছে, কিন্তু বড় অ্যাপে performance issue হতে পারে। Provider বা Bloc ব্যবহার করা ভালো।

6. Testing

Q: ইউনিট টেস্ট করেছো?

- **A:** এখনও করিনি, তবে Firebase এর সাথে Mock ব্যবহার করে টেস্ট করা যায়।

Q: Widget টেস্ট কীভাবে করবে?

- **A:** flutter test ব্যবহার করে widget টেস্ট করা যায়, Provider এর জন্য ProviderContainer ব্যবহার করতে হবে।

7. Edge Cases

Q: ইউজার যদি ইন্টারনেট ছাড়া অ্যাপ ব্যবহার করে?

- A: SharedPreferences এ ক্যাশ রাখা আছে, কিন্তু Firestore অপারেশন fail হবে। offline support এর জন্য Firebase Offline Persistence enable করা যায়।

Q: একই ডেটা দুবার সেভ হলে কী হয়?

- A: ID unique রাখা হয়েছে (UUID), তাই duplicate হবে না।

কোড Change করলে কী হয়?

Change 1: যদি Steps Screen এ `_stepGoal` এ 5000 সেট করি

```
dart
int _stepGoal = 5000;
```

- **Output:** Progress bar 5000 এর ভিত্তিতে দেখাবে, আগে 10000 ছিল।

Change 2: যদি Gemini এর temperature 1.0 করি

```
dart
temperature: 1.0,
```

- **Output:** AI আরও ক্রিয়েটিভ এবং র্যান্ডম উত্তর দেবে, কম consistent.

Change 3: যদি Firestore এর `getTotalWorkouts()` এ `.get()` এর পরিবর্তে `.snapshots()` ব্যবহার করি

```
dart
Stream<int> getTotalWorkouts() {
```

```
return _db.collection('users').snapshots().map(...);
}
```

- **Output:** রিয়েল-টাইম আপডেট পাবে, কিন্তু বেশি cost হবে।

Defence ভাইভাতে কী কী প্রশ্ন আসতে পারে?

1. **প্রকল্পের উদ্দেশ্য কী?** → ফিটনেস ট্র্যাকিং + AI চ্যাটবট
2. **কেন ফ্লাটার?** → Cross-platform, fast development
3. **Firebase কেন?** → Real-time database, authentication, easy setup
4. **State management কী ব্যবহার করেছো?** → Provider
5. **কীভাবে ডেটা মডেল করেছো?** → User → sub-collections
6. **Security Rules কী?** → শুধু authenticated user নিজের ডেটা access করতে পারে
7. **Performance optimize করেছো?** → StreamBuilder, FutureBuilder, limit queries
8. **Offline support আছে?** → SharedPreferences ব্যবহার করেছি
9. **Testing করেছো?** → এখনও করিনি, future scope
10. **কী improve করতে পারো?** → Backend API, Push Notification, Pagination, Better UI/UX

সারাংশ

এই প্রজেক্টটি একটি ফুল-স্ট্যাক ফ্লাটার অ্যাপ্লিকেশন যেখানে:

- **Firebase** ব্যাকএন্ড হিসেবে কাজ করে
- **Gemini AI** চ্যাটবট ফিচার দেয়
- **Provider** state management করে
- **CustomPainter** দিয়ে সুন্দর UI তৈরি করা হয়েছে

ভাইভাতে কনফিডেন্ট থাকার জন্য:

1. প্রতিটি widget এর purpose জানতে হবে
2. Firebase এর basic operations জানতে হবে
3. State management এর logic বুঝতে হবে
4. Error handling সম্পর্কে জানতে হবে
5. Performance optimization এর কথা বলতে হবে

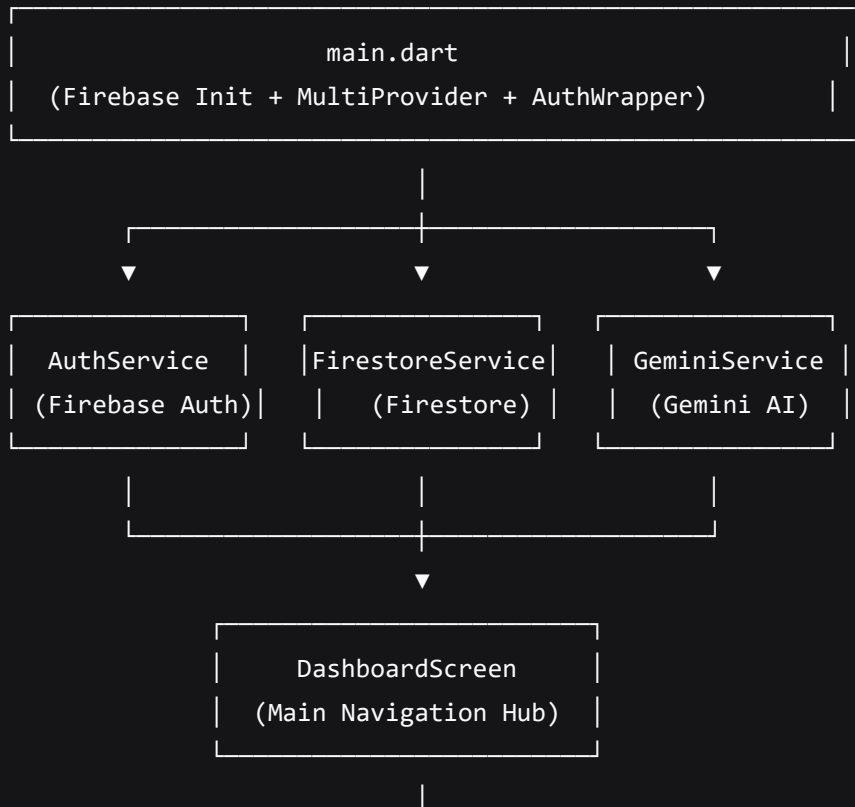
শুভকামনা! 🙏

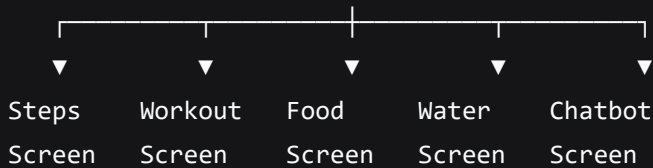
akhane prottek ta code explain kore daw vivar jonno abong viva te kuthay code change korte bolbe jate ui te change hoy ta explain kore daw

ফিটনেস অ্যাপ্লিকেশন - সম্পূর্ণ কোড ব্যাখ্যা ও ভাইভা প্রস্তুতি

প্রকল্পের আর্কিটেকচার (প্রথমে বুঝে নিন)

text





1. main.dart - এন্ট্রি পয়েন্ট

dart

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized(); // Flutter ইঞ্জিন ইনিশিয়ালাইজ  
  await Firebase.initializeApp();           // Firebase সেটআপ  
  runApp(const MyApp());  
}
```

কেন এই কোড?

- `WidgetsFlutterBinding.ensureInitialized()` - Firebase এর আগে Flutter ইঞ্জিন রেডি করতে
- `Firebase.initializeApp()` - Firebase কনফিগারেশন লোড করতে

dart

```
MultiProvider(  
  providers: [  
    ChangeNotifierProvider(create: (_) => ThemeProvider()), // থিম পরিবর্তন  
    Provider<AuthService>(create: (_) => AuthService()),      // Authentication  
    Provider<FirestoreService>(create: (_) => FirestoreService()), // Database  
    Provider<GeminiService>(create: (_) => GeminiService()), // AI Service  
    ProxyProvider2<AuthService, FirestoreService, StepService?>(  
      update: (_, auth, firestore, previous) {  
        final user = auth.currentUser;  
        if (user != null) {  
          return previous ?? StepService(firestore, user.uid);  
        }  
        return null;  
      },  
    ),  
  ],  
)
```

)

ভাইভা প্রশ্ন:

- **Q: ProxyProvider2 কেন ব্যবহার করেছো?**
- A: StepService এর জন্য AuthService এবং FirestoreService উভয়ের উপর নির্ভরশীল। ইউজার লগইন করলেই StepService তৈরি হবে।

```
dart
class AuthWrapper extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: authService.authStateChanges,
      builder: (context, snapshot) {
        if (user == null) return LoginScreen();
        else if (user.emailVerified) return DashboardScreen();
        else return VerifyEmailScreen();
      },
    );
  }
}
```

ফ্লো:

1. ইউজার লগইন না করলে → LoginScreen
2. লগইন করলে কিন্তু email verified না → VerifyEmailScreen
3. লগইন + email verified → DashboardScreen

2. app_theme.dart - থিম কনফিগারেশন

```
dart
static const Color primaryColor = Color(0xFF7F9A00); // সবুজাভ হলুদ
static const Color secondaryColor = Color(0xFF00D1FF); // সায়ান
static const Color accentColor = Color(0xFFFF0055); // গোলাপী
```

```
static ThemeData get darkTheme {
  return ThemeData(
    useMaterial3: true, // Material 3 ডিজাইন সিস্টেম
    brightness: Brightness.dark,
    textTheme: GoogleFonts.outfitTextTheme(...), // Outfit ফন্ট
  );
}
```

ভাইভা প্রশ্ন:

- **Q: GoogleFonts কেন ব্যবহার করেছো?**
- A: অ্যাপকে ইউনিক এবং প্রফেশনাল লুক দিতে
- **Q: Material 3 কী?**
- A: Google এর নতুন ডিজাইন সিস্টেম, Dynamic Color সাপোর্ট

3. auth_service.dart - অথেনটিকেশন

```
dart
class AuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Stream<User?> get authStateChanges => _auth.authStateChanges();
  User? get currentUser => _auth.currentUser;

  Future<UserCredential> signIn(String email, String password) async {
    return await _auth.signInWithEmailAndPassword(
      email: email,
      password: password
    );
  }
}
```

কেন আলাদা Service?

- Separation of Concern - UI থেকে Auth logic আলাদা
- Reusability - যেকোনো Screen থেকে call করা যায়

4. firestore_service.dart - ডেটাবেস অপারেশন

```
dart
// User ডেটা সেভ
Future<void> saveUser(UserModel user) async {
  await _db.collection('users').doc(user.id).set(user.toMap());
}

// Realtime Stream - Workouts
Stream<List<WorkoutModel>> getWorkouts(String uid) {
  return _db
    .collection('users')
    .doc(uid)
    .collection('workouts')
    .orderBy('timestamp', descending: true) // লেটেস্ট প্রথমে
    .snapshots() // Realtime updates
    .map((snapshot) => snapshot.docs
      .map((doc) => WorkoutModel.fromMap(doc.data()))
      .toList());
}
```

ভাইভা প্রশ্ন:

- **Q:** `.snapshots()` এবং `.get()` এর মধ্যে পার্থক্য কী?
 - `.snapshots()` → Realtime updates (Stream)
 - `.get()` → একবার ডেটা নেয় (Future)
- **Q: Sub-collection কেন ব্যবহার করেছো?**
 - A: ডেটা সংগঠিত রাখতে এবং Security Rules সহজ করতে

```
dart
// Admin: সব ইউজারের ডেটা এক্সেস
```

```

Stream<List<UserModel>> getAllUsers() {
  return _db.collection('users').snapshots().map(...);
}

// Admin: টোটাল ওয়ার্কআউট কাউন্ট (N+1 Query Problem)
Future<int> getTotalWorkouts() async {
  int total = 0;
  final users = await _db.collection('users').get();
  for (var user in users.docs) {
    final workouts = await _db
      .collection('users')
      .doc(user.id)
      .collection('workouts')
      .get();
    total += workouts.docs.length;
  }
  return total;
}

```

⚠ **সমস্যা:** N+1 Query Problem - প্রতিটি ইউজারের জন্য আলাদা Query

সমাধান: Firebase Aggregation বা Counter Field ব্যবহার করা

5. gemini_service.dart - AI ইন্টিগ্রেশন

```

dart
GeminiService() {
  _model = GenerativeModel(
    model: 'gemini-2.5-flash',
    apiKey: _apiKey,
    generationConfig: GenerationConfig(
      temperature: 0.7, // 0-1, বেশি হলে ক্রিয়েটিভ
      topK: 40,
      topP: 0.95,
      maxOutputTokens: 1024, // রেসপন্সের দৈর্ঘ্য
    ),
  ),

```

```
);  
}
```

ভাইভা প্রশ্ন:

- **Q: temperature 0.7 কেন?**
- A: Balanced - খুব র‍্যাবলম না, খুব ডিটারমিনিস্টিক না
- **Q: API Key client-side এ রাখা কি safe?**
- A: না! Production এ Firebase Functions বা Backend ব্যবহার করা উচিত

```
dart  
// Chat History  
_chat = _model.startChat(  
  history: [  
    Content.text('You are a helpful fitness coach...'), // System Prompt  
  ],  
);
```

System Prompt এর কাজ: AI কে কীভাবে আচরণ করতে হবে তা বলা

6. dashboard_screen.dart - হোম স্ক্রিন

```
dart  
SliverAppBar.large( // Expandable AppBar  
  actions: [  
    // Admin Button (শুধু অ্যাডমিন দেখবে)  
    FutureBuilder<UserModel?>(  
      future: firestoreService.getUser(user.uid),  
      builder: (context, snapshot) {  
        if (snapshot.hasData && snapshot.data?.isAdmin == true) {  
          return IconButton(  
            icon: const Icon(Icons.admin_panel_settings),  
            onPressed: () => Navigator.push(...),  
          );  
        }  
      }  
    )  
  ]  
);
```

```

        return const SizedBox.shrink(); // Empty widget
      },
    ),
  ],
)
)

dart
// Weekly Workout Chart
SizedBox(
  height: 220,
  child: StreamBuilder<List<double>>(
    stream: firestoreService.getWeeklyWorkoutData(user.uid),
    builder: (context, snapshot) {
      return BarChart(
        BarChartData(
          barGroups: List.generate(7, (index) {
            return makeGroupData(index, weekData[index], ...);
          }),
        ),
      );
    },
  ),
)
)

```

fl_chart ব্যবহারের কারণ: কাস্টমাইজেবল চার্ট, সুন্দর UI

7. login_screen.dart & register_screen.dart

```

dart
class LoginScreen extends StatefulWidget {
  final _formKey = GlobalKey<FormState>(); // Form Validation
}

TextFormField(
  controller: _emailController,
  validator: Validators.validateEmail, // Email validation
)

```

```

)

void _login() async {
  if (_formKey.currentState!.validate()) { // Check validation
    await authService.signIn(email, password);
  }
}

```

Validation Flow:

1. ইউজার ইনপুট দেয়
2. `_formKey.currentState!.validate()` চেক করে
3. Valid না হলে error message দেখায়
4. Valid হলে `signIn()` call করে

8. validators.dart - ইনপুট ভ্যালিডেশন

```

dart
static final RegExp _emailRegex = RegExp(
  r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
);

static String? validateEmail(String? value) {
  if (value == null || value.isEmpty) return 'Please enter email';
  if (!_emailRegex.hasMatch(value)) return 'Invalid email';
  return null; // Valid
}

```

Regex এর কাজ: Email এর প্যাটার্ন চেক করা

9. steps_screen.dart - স্টেপ ট্র্যাকার

```
dart
class _CircularProgressPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    // Background Circle
    canvas.drawCircle(center, radius, bgPaint);

    // Progress Arc
    final sweepAngle = 2 * pi * progress;
    canvas.drawArc(rect, -pi/2, sweepAngle, false, progressPaint);
  }
}
```

CustomPainter কেন?

- Flutter এর built-in widget ছিল না কাস্টম সার্কুলার প্রোগ্রেস বার
- সম্পূর্ণ কন্ট্রোল পেতে

```
dart
AnimatedBuilder(
  animation: _progressAnim,
  builder: (context, child) {
    return CustomPaint(
      painter: _CircularProgressPainter(
        progress: progress * _progressAnim.value,
      ),
    );
  },
)
```

Animation Flow:

1. `AnimationController` - 1200ms duration
2. `CurvedAnimation` - easeOutCubic curve
3. `AnimatedBuilder` - প্রতিটি frame এ rebuild

10. chatbot_screen.dart - AI চ্যাটবট

```
dart
void _sendMessage() async {
  // 1. User message যোগ করুন
  _messages.add(userMessage);

  // 2. AI response নিন
  final response = await _geminiService.sendMessage(userMessage.text);

  // 3. AI message যোগ করুন
  _messages.add(aiMessage);

  // 4. Firestore এ সেভ করুন
  final chatLog = ChatLogModel(...);
  firestore.saveChatLog(chatLog);
}

dart
Widget _buildMessageBubble(ChatMessage message) {
  return Row(
    mainAxisAlignment: isUser ? MainAxisAlignment.end : MainAxisAlignment.start,
    children: [
      if (!isUser) CircleAvatar(...), // AI avatar
      Flexible(child: Container(...)), // Message bubble
      if (isUser) CircleAvatar(...), // User avatar
    ],
  );
}
```

Message Bubble Design:

- User → ডান পাশে, Gradient background
- AI → বাম পাশে, Card background

11. food_tracker_screen.dart - ক্যালোরি ট্র্যাকার

```
dart
StreamBuilder<double>(
  stream: firestoreService.getTodayCalories(user.uid),
  builder: (context, snapshot) {
    final calories = snapshot.data ?? 0;
    return Text('${calories.toStringAsFixed(0)} kcal');
  },
)

dart
Map<DateTime, List<FoodIntake>> _groupLogsByDate(List<FoodIntake> logs) {
  final grouped = {};
  for (final log in logs) {
    final date = DateTime(log.timestamp.year, log.timestamp.month, log.timestamp.day);
    ;
    grouped.putIfAbsent(date, () => []).add(log);
  }
  return grouped;
}
```

Grouping কেন? ডেটা তারিখ অনুযায়ী সাজানো

12. food_input_screen.dart - ফুড ইনপুট

```
dart
Future<void> _searchAI(String query) async {
  final calories = await gemini.getCaloriesForFood(query);
  if (calories != null) {
    final customItem = FoodItem(
      name: query.trim(),
      caloriesPerServing: calories,
      servingSize: '1 serving',
    );
    _searchResults = [customItem, ..._searchResults];
  }
}
```

```
}  
}
```

AI Integration: ইউজার যেকোনো খাবারের ক্যালোরি জানতে পারে

13. profile_screen.dart - প্রোফাইল

```
dart  
SwitchListTile(  
  title: const Text('Dark Mode'),  
  value: isDark,  
  onChanged: (val) => themeProvider.toggleTheme(val),  
)
```

ThemeProvider:

```
dart  
class ThemeProvider with ChangeNotifier {  
  ThemeMode _themeMode = ThemeMode.system;  
  
  void toggleTheme(bool isDark) {  
    _themeMode = isDark ? ThemeMode.dark : ThemeMode.light;  
    notifyListeners(); // UI update  
  }  
}
```

14. theme_provider.dart - থিম ম্যানেজমেন্ট

```
dart  
Future<void> _loadTheme() async {  
  final prefs = await SharedPreferences.getInstance();  
  final isDark = prefs.getBool('isDarkMode');  
  _themeMode = isDark ? ThemeMode.dark : ThemeMode.light;
```

```
notifyListeners();  
}
```

SharedPreferences: ইউজারের পছন্দ সংরক্ষণ করে

15. notification_service.dart - নোটিফিকেশন

```
dart  
tz.TZDateTime _nextInstanceOf8AM() {  
  final now = tz.TZDateTime.now(tz.local);  
  return tz.TZDateTime(tz.local, now.year, now.month, now.day, 8);  
}
```

timezone package: সঠিক লোকাল টাইমে নোটিফিকেশন শিডিউল করতে

ভাইভাতে কোড পরিবর্তনের প্রশ্ন ও উত্তর

প্রশ্ন ১: "Dashboard এ কালার থিম পরিবর্তন করতে বললে কী করবে?"

```
dart  
// app_theme.dart এ পরিবর্তন  
static const Color primaryColor = Color(0xFF7F9A00);  
// পরিবর্তন করে:  
static const Color primaryColor = Color(0xFFFF6B6B); // লাল
```

আউটপুট: সব বাটন, হেডার, আইকন লাল হবে

প্রশ্ন ২: "Steps Screen এ Step Goal 10000 থেকে 8000 করলে?"

```
dart
// steps_screen.dart
int _stepGoal = 10000;
// পরিবর্তন:
int _stepGoal = 8000;
```

আউটপুট: Progress bar 8000 এর ভিত্তিতে দেখাবে, 100% পেতে 8000 স্টেপ লাগবে

প্রশ্ন ৩: "Gemini temperature 0.7 থেকে 1.0 করলে?"

```
dart
// gemini_service.dart
temperature: 0.7,
// পরিবর্তন:
temperature: 1.0,
```

আউটপুট: AI আরও ক্রিয়েটিভ, ভিন্ন ভিন্ন উত্তর দেবে, কম consistent

প্রশ্ন ৪: "Validation এ পাসওয়ার্ডের নিয়ম পরিবর্তন করলে?"

```
dart
// validators.dart
static final RegExp _passwordRegex = RegExp(
  r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{6,}$',
);
// পরিবর্তন (শুধু 8+ characters):
static final RegExp _passwordRegex = RegExp(
  r'^.{8,}$',
);
```

আউটপুট: ইউজার "password123" লিখতে পারবে (Uppercase/Number ছাড়া)

প্রশ্ন ৫: "Food Database এ নতুন ফুড যোগ করলে?"

```
dart
// food_database.dart
static const List<FoodItem> foods = [
  // নতুন যোগ:
  FoodItem(
    name: 'Mango Smoothie',
    caloriesPerServing: 250,
    servingSize: '1 cup',
  ),
];
```

আউটপুট: সার্চ করলে "Mango Smoothie" দেখাবে

প্রশ্ন ৬: "Admin Dashboard এ Total Users এর কালার পরিবর্তন করলে?"

```
dart
// admin_dashboard_screen.dart
return _buildStatCard(
  color: Colors.blue, // পরিবর্তন:
  color: Colors.red,
);
```

আউটপুট: Total Users কার্ডের আইকন এবং গ্রেডিয়েন্ট লাল হবে

প্রশ্ন ৭: "Chatbot এ Welcome Message পরিবর্তন করলে?"

```
dart
// chatbot_screen.dart
_messages.add(ChatMessage(
  text: 'Hello! I\'m your AI Fitness Coach...',
));
// পরিবর্তন:
_messages.add(ChatMessage(
  text: 'Welcome to Fit Axis! How can I help you today?',
));
```

আউটপুট: চ্যাট খুললেই নতুন মেসেজ দেখাবে

প্রশ্ন ৮: "BMI Calculator এ Height এর Range পরিবর্তন করলে?"

```
dart
// bmi_screen.dart
Slider(
  min: 100, max: 220, // পরিবর্তন:
  min: 80, max: 250,
)
```

আউটপুট: ইউজার 80-250 cm পর্যন্ত height দিতে পারবে

ভাইভাতে গুরুত্বপূর্ণ প্রশ্নসমূহ

Architecture & Design

1. Q: কেন Provider ব্যবহার করেছো? Bloc বা Riverpod কেন না?

- A: প্রজেক্ট ছোট, Provider lightweight এবং সহজ। Dependency injection এর জন্য ভালো।
2. **Q: Service Pattern কী?**
- A: Business logic UI থেকে আলাদা করার প্যাটার্ন। AuthService, FirestoreService ইত্যাদি।
3. **Q: Separation of Concern কীভাবে maintain করেছো?**
- A: UI (Screens), Models (Data), Services (Logic) আলাদা folder এ রেখেছি।

Firestore

4. **Q: Firestore এ ডেটা কীভাবে মডেল করেছো?**

- A: Users → sub-collections (workouts, steps, food, water, chats)

5. **Q: Security Rules কীভাবে সেট করবে?**

- A:

javascript

```
match /users/{userId}/{document=**} {  
  allow read, write: if request.auth.uid == userId;  
}  
match /users/{userId} {  
  allow read: if request.auth.token.admin == true;  
}
```

6. **Q: N+1 Query Problem কী? কোডে কোথায় আছে?**

- A: getTotalWorkouts() এ - প্রথমে সব user, তারপর প্রতিটি user এর workouts আলাদা query

State Management

7. **Q: setState() কোথায় ব্যবহার করেছো? সমস্যা কী?**

- A: ছোট widgets এ (AddStepForm, BMI Calculator)। বড় অ্যাপে performance issue হতে পারে।

8. **Q: StreamBuilder vs FutureBuilder?**

- A: `StreamBuilder` → Realtime updates (`getWorkouts`), `FutureBuilder` → একবার ডেটা (`getTotalWorkouts`)

Performance

9. **Q: Performance optimize করেছো কীভাবে?**

- A:
- `.limit()` ব্যবহার করিনি (ভবিষ্যতে করতে হবে)
- `StreamBuilder` এর `initialData` ব্যবহার
- `const` widgets ব্যবহার

10. **Q: অ্যাপে memory leak আছে?**

- A: `dispose()` method এ controllers এবং `AnimationController` dispose করেছি

AI Integration

11. **Q: Gemini API এর rate limit কীভাবে handle করবে?**

- A: try-catch, retry mechanism, অথবা API call এর মধ্যে delay

12. **Q: API Key security?**

- A: Production এ Firebase Functions বা Backend ব্যবহার করা উচিত

Testing

13. **Q: Unit test করেছো?**

- A: এখনও করিনি। `Validators.dart` এর জন্য test করা যায়।

14. **Q: Widget test কীভাবে করবে?**

- A: `flutter test` + `ProviderContainer` ব্যবহার করে

Edge Cases

15. **Q: Internet disconnected হলে কী হয়?**

- A: Firestore offline support আছে (default), কিন্তু SharedPreferences এ ক্যাশ আছে

16. **Q: একই ডেটা দুবার সেভ হলে?**

- A: UUID ব্যবহার করেছি, তাই unique

প্র্যাকটিক্যাল টাস্ক (ভাইভাতে করতে বলতে পারে)

টাস্ক ১: নতুন ফিচার যোগ করা - "Daily Quote"

```
dart
// dashboard_screen.dart এ যোগ
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        Text('Daily Motivation', style: TextStyle(fontWeight: FontWeight.bold)),
        Text('"The only bad workout is the one that didn\'t happen."'),
      ],
    ),
  ),
)
```

টাস্ক ২: Error Handling improve করা

```
dart
// firestore_service.dart এর getTotalWorkouts() এ
Future<int> getTotalWorkouts() async {
  try {
    // existing code
  } catch (e) {
    print('Error getting total workouts: $e');
  }
}
```

```
    return 0;
  }
}
```

টাস্ক ৩: Loading State যোগ করা

```
dart
// যেকোনো FutureBuilder এ
if (snapshot.connectionState == ConnectionState.waiting) {
  return const Center(child: CircularProgressIndicator());
}
```

সংক্ষিপ্ত ভাইভা চিটশিট

টপিক	কী বলবেন
প্রকল্পের উদ্দেশ্য	ফিটনেস ট্র্যাকিং + AI চ্যাটবট
ব্যবহৃত টেকনোলজি	Flutter, Firebase, Gemini AI, Provider
State Management	Provider (ChangeNotifier)
Database	Firestore (NoSQL)
Authentication	Firebase Auth (Email/Password)
AI	Google Gemini (gemini-2.5-flash)
Chart	fl_chart
Local Storage	SharedPreferences
Testing	এখনও করিনি (Future Scope)

টপিক

কী বলবেন

Performance

StreamBuilder, const widgets, dispose()

Security

API Key client-side (improve needed)